

Experiment No 1

Title:

Perform tokenization (Whitespace, Punctuation-based, Treebank, Tweet, MWE) using NLTK library. Use porter stemmer and snowball stemmer for stemming. Use any technique for lemmatization

Objective:

- To understand and implement different tokenization methods: Whitespace, Punctuation-based, Treebank, Tweet, and Multi-Word Expression (MWE) tokenization
- To compare Porter Stemmer and Snowball Stemmer for word stemming
- To perform lemmatization using WordNet Lemmatizer with part-of-speech tagging
- To analyze the differences between stemming and lemmatization techniques

Course Outcomes:

- CO1: Understand fundamental text preprocessing techniques in Natural Language Processing
- CO2: Apply tokenization, stemming, and lemmatization algorithms using Python libraries
- CO3: Analyze and compare different NLP preprocessing methods for text normalization

PO & PSO Mapping

PO/PSO	Description	Level
PO1	Apply knowledge of computing fundamentals for NLP tasks	3
PO2	Design and implement text preprocessing solutions	3
PO3	Analyze experimental results and draw conclusions	2
PSO1	Demonstrate proficiency in Python programming for AI/ML	3

PO/PSO	Description	Level
	applications	
PSO2	Apply data science techniques for text analytics	2

Hardware Requirements:

- System: Intel Core i3 or higher processor
- RAM: Minimum 4 GB (8 GB recommended)
- Hard Disk: Minimum 20 GB free space
- Display: Standard monitor with minimum 1024×768 resolution

Software Requirements:

- Operating System: Windows 10/11, Linux (Ubuntu 18.04 or higher), or macOS
- Python: Version 3.7 or higher
- IDE: Jupyter Notebook, PyCharm, VS Code, or Google Colab
- Libraries: NLTK 3.5 or higher, NumPy, Pandas

Theory

1. Tokenization

Tokenization is the process of breaking down text into smaller units called tokens (words, phrases, or symbols). It is the foundational step in text preprocessing for NLP applications.

Types of Tokenization:

a) Whitespace Tokenization: Splits text based on whitespace characters (spaces, tabs, newlines). Simple but doesn't handle punctuation effectively.

b) Punctuation-based Tokenization (WordPunct): Separates words and punctuation marks into individual tokens. Treats punctuation as separate tokens.

c) Treebank Tokenization: Uses regular expressions following Penn Treebank conventions. Handles contractions (e.g., "can't" → "ca", "n't"), separates punctuation intelligently, and preserves decimal numbers as single tokens.

d) Tweet Tokenization: Specialized for social media text. Preserves hashtags, mentions (@username), emoticons, URLs, and handles repeated characters.

e) Multi-Word Expression (MWE) Tokenization: Treats predefined multi-word phrases as single tokens. Useful for named entities like "New York" or technical terms like "Machine Learning".

2. Stemming

Stemming reduces words to their root or base form by removing suffixes using heuristic rules. It's a crude chopping process that may not always produce valid words.

a) Porter Stemmer: The most widely used stemming algorithm. Applies a series of rules to strip common suffixes. For example: "running" → "run", "generously" → "generous".

b) Snowball Stemmer: Also known as Porter2 stemmer. An improved and more aggressive version of Porter Stemmer. Supports multiple languages and produces more accurate stems.

3. Lemmatization

Lemmatization reduces words to their dictionary base form (lemma) using vocabulary and morphological analysis. Unlike stemming, it always produces valid words and considers the context (part of speech).

Algorithm / Procedure:

Step 1: Import necessary libraries

- Import NLTK library
- Import tokenizers: WhitespaceTokenizer, WordPunctTokenizer, TreebankWordTokenizer, TweetTokenizer, MWETokenizer
- Import stemmers: PorterStemmer, SnowballStemmer
- Import lemmatizer: WordNetLemmatizer

Step 2: Download NLTK resources

- Download 'punkt' for sentence tokenization
- Download 'wordnet' for lemmatization
- Download 'averaged_perceptron_tagger' for POS tagging
- Download 'omw-1.4' for multilingual wordnet

Step 3: Perform Tokenization

- Create sample text with various elements (punctuation, hashtags, URLs)
- Initialize each tokenizer
- Apply tokenization and display results
- Compare token counts and token formats

Step 4: Perform Stemming

- Define a list of words with different forms
- Initialize Porter Stemmer
- Apply stemming and record results
- Initialize Snowball Stemmer
- Apply stemming and compare with Porter results

Step 5: Perform Lemmatization

- Define a list of words for lemmatization
- Initialize WordNet Lemmatizer
- Apply lemmatization with default POS (noun)
- Apply lemmatization with specific POS tags (verb, adjective, adverb)
- Compare results with stemming output

Step 6: Analyze and compare results

- Compare tokenization outputs
- Compare stemming vs lemmatization
- Document observations

Viva Questions

- What is tokenization and why is it important in NLP?
- What is the difference between stemming and lemmatization?
- Name three types of tokenizers available in NLTK.
- What is the purpose of the Porter Stemmer?
- What does MWE stand for in tokenization?

Observation:

Tokenization Method	Sample Input	Token Count	Handles Punctuation	Handles Hashtags/URLs
Whitespace	"I'm learning #NLP"	3	No	No
Punctuation-based	"I'm learning #NLP"	6	Yes	Partially
Treebank	"I can't wait"	5	Yes	No

Tokenization Method	Sample Input	Token Count	Handles Punctuation	Handles Hashtags/URLs
Tweet	"#NLP @user :)"	3	Yes	Yes
MWE	"New York city"	2 (with MWE)	No	No

Stemming Algorithm	Word: "running"	Word: "generously"	Word: "fairness"
Porter Stemmer	run	generous	fair
Snowball Stemmer	run	generous	fair

Technique	Word: "running" (verb)	Word: "better" (adj)	Produces Valid Words
Stemming	run	better	Not always
Lemmatization	run	good	Always

Result / Conclusion

The experiment successfully demonstrated various text preprocessing techniques using NLTK library.

Whitespace tokenization is simplest but limited for complex text with punctuation Tweet tokenizer effectively preserves social media elements like hashtags and mentions Treebank tokenization intelligently handles contractions and punctuation following Penn Treebank standards MWE tokenization successfully groups multi-word expressions into single tokens Snowball Stemmer is more aggressive than Porter Stemmer and produces slightly different results for certain words Lemmatization with POS tags produces more accurate base forms compared to stemming, always returning valid dictionary words Stemming is faster but less accurate, while lemmatization is slower but linguistically more accurate

Experiment No 2

Title:

To implement and analyze text vectorization techniques including Bag-of-Words (count occurrence and normalized count), TF-IDF (Term Frequency-Inverse Document Frequency), and Word2Vec embeddings for converting text data into numerical representations

Objective:

- To understand and implement Bag-of-Words model with count occurrence and normalized count occurrence
- To compute TF-IDF scores for text documents and understand their significance
- To create word embeddings using Word2Vec model (CBOW and Skip-gram)
- To compare different text representation techniques and their applications
- To analyze word similarities and relationships using vector embeddings

Course Outcomes:

- CO1: Understand fundamental feature extraction techniques in Natural Language Processing
- CO2: Apply text vectorization algorithms for machine learning applications
- CO3: Evaluate and compare different text representation methods
- CO4: Implement word embeddings for semantic similarity analysis

PO & PSO Mapping

PO/PSO	Description	Level
PO1	Apply knowledge of mathematics and computing for feature extraction	3
PO2	Design solutions for text analytics problems	3
PO3	Analyze and interpret vectorized text representations	3
PO5	Use modern tools like scikit-learn and gensim for NLP	3
PSO1	Demonstrate proficiency in Python for machine learning	3
PSO2	Apply deep learning concepts (Word2Vec) for text processing	3

Hardware Requirements:

- System: Intel Core i3 or higher processor
- RAM: Minimum 4 GB (8 GB recommended)
- Hard Disk: Minimum 20 GB free space
- Display: Standard monitor with minimum 1024×768 resolution

Software Requirements:

- Operating System: Windows 10/11, Linux (Ubuntu 18.04 or higher), or macOS
- Python: Version 3.7 or higher
- IDE: Jupyter Notebook, PyCharm, VS Code, or Google Colab
- Libraries: scikit-learn 1.0 or higher, gensim 4.0 or higher, NLTK 3.6 or higher
NumPy, Pandas, Matplotlib

Theory

1. Bag-of-Words (BoW) Model

Bag-of-Words is a text representation technique that converts text documents into numerical feature vectors by counting word occurrences. It treats each document as an unordered collection of words, ignoring grammar and word order but maintaining word frequency information.

Working Principle:

- Create a vocabulary of all unique words from the entire corpus
- For each document, count the occurrence of each vocabulary word
- Represent each document as a vector of word counts

a) Count Occurrence: Creates a document-term matrix where each cell contains the raw count of how many times a word appears in a document.

b) Normalized Count Occurrence: Normalizes the count vectors (typically using L1 or L2 normalization) to account for document length differences. This prevents longer documents from having artificially higher feature values.

Advantages:

- Simple to understand and implement
- Works well for text classification tasks
- Computationally efficient

Disadvantages:

- Ignores word order and context
- Creates high-dimensional sparse vectors
- Doesn't capture semantic meaning

2. TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF is an advanced text vectorization technique that weighs words based on their importance in a document relative to the entire corpus. It addresses the limitation of BoW by reducing the weight of common words.

Components:

Term Frequency (TF): Measures how frequently a term appears in a document

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

Inverse Document Frequency (IDF): Measures how important a term is across all documents

$$IDF(t) = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents containing term } t} \right)$$

TF-IDF Score:

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

Interpretation:

- High TF-IDF: Word appears frequently in a document but rarely in others (important/distinctive)
- Low TF-IDF: Word appears rarely in a document or commonly across all documents

3. Word2Vec Embeddings

Word2Vec is a neural network-based technique that creates dense vector representations of words by learning from their context in large text corpora. Unlike BoW and TF-IDF, Word2Vec captures semantic relationships between words.

Architectures:

a) CBOW (Continuous Bag-of-Words): Predicts a target word from its surrounding context words. Faster and works well with frequent words.

b) Skip-gram: Predicts surrounding context words from a target word. Works better with rare words and smaller datasets.

Key Parameters:

- vector_size: Dimensionality of word vectors (typically 50-300)
- window: Maximum distance between current and predicted word
- min_count: Minimum word frequency threshold
- eg: Training algorithm (0 for CBOW, 1 for Skip-gram)

Advantages:

- Captures semantic and syntactic relationships
- Creates dense, low-dimensional representations
- Enables word similarity and analogy tasks

- Preserves word meaning in vector space

Word Similarity: Words with similar meanings have similar vector representations, measured using cosine similarity.

Algorithm / Procedure:

Step 1: Import required libraries

- Import sklearn.feature_extraction.text (CountVectorizer, TfidfVectorizer)
- Import gensim.models (Word2Vec)
- Import numpy for numerical operations

Step 2: Prepare text corpus

- Create or load text documents
- Store documents in a list
- Display corpus for verification

Step 3: Implement Bag-of-Words (Count Occurrence)

- Initialize CountVectorizer
- Fit and transform the corpus
- Extract vocabulary and feature names
- Generate document-term matrix
- Display count occurrence results

Step 4: Implement Normalized Count Occurrence

- Convert count matrix to numpy array
- Apply L1 normalization (divide by row sum)
- Display normalized values
- Compare with raw counts

Step 5: Implement TF-IDF

- Initialize TfidfVectorizer
- Fit and transform the corpus
- Extract TF-IDF scores
- Display IDF values for vocabulary
- Analyze most important terms per document

Step 6: Create Word2Vec Embeddings

- Tokenize corpus into sentences/words
- Initialize Word2Vec model with parameters
- Train model on tokenized corpus
- Extract word vectors
- Perform similarity analysis
- Find most similar words

Step 7: Analyze and compare results

- Compare representation sizes (sparse vs dense)
- Analyze computational efficiency
- Evaluate semantic preservation
- Document observations

Viva Questions

- What is the Bag-of-Words model and how does it represent text?
- What is the difference between count occurrence and normalized count occurrence?
- What does TF-IDF stand for?
- What is the purpose of IDF in TF-IDF?
- Name two Word2Vec architectures.

Observation:

Technique	Representation Type	Vector Dimensions	Sparsity	Captures Semantics	Computational Cost
BoW (Count)	Sparse	High (vocab size)	Very High	No	Low
BoW (Normalized)	Sparse	High (vocab size)	Very High	No	Low
TF-IDF	Sparse	High (vocab size)	High	Partial	Medium
Word2Vec	Dense	Low (50-300)	Very Low	Yes	High

Result / Conclusion

The experiment successfully implemented three major text vectorization techniques: Bag-of-Words, TF-IDF, and Word2Vec embeddings

Experiment No 3

Title:

To perform comprehensive text preprocessing including text cleaning, lemmatization, stop words removal, label encoding, and create TF-IDF representations for text classification tasks.

Objective:

- To clean raw text data by removing noise, punctuation, numbers, and special characters
- To apply lemmatization for reducing words to their base forms
- To remove stop words to focus on meaningful content
- To encode categorical labels into numerical format using Label Encoding
- To create TF-IDF feature representations for machine learning models
- To save all processed outputs for future use in classification tasks

Course Outcomes:

- CO1: Understand complete text preprocessing pipeline for NLP applications
- CO2: Apply text cleaning and normalization techniques systematically
- CO3: Implement feature extraction using TF-IDF vectorization
- CO4: Prepare data for machine learning classification tasks
- CO5: Save and serialize machine learning models and preprocessing objects

PO & PSO Mapping

PO/PSO	Description	Level
PO1	Apply fundamental preprocessing concepts for text analytics	3
PO2	Design complete data preprocessing pipelines	3
PO3	Analyze and evaluate preprocessing quality	3
PO5	Use modern tools (sklearn, NLTK, pickle) for NLP	3
PSO1	Demonstrate proficiency in Python for data preprocessing	3
PSO2	Apply machine learning preprocessing techniques	3

Hardware Requirements:

- System: Intel Core i3 or higher processor
- RAM: Minimum 4 GB (8 GB recommended)
- Hard Disk: Minimum 20 GB free space
- Display: Standard monitor with minimum 1024×768 resolution

Software Requirements:

- Operating System: Windows 10/11, Linux (Ubuntu 18.04 or higher), or macOS
- Python: Version 3.7 or higher
- IDE: Jupyter Notebook, PyCharm, VS Code, or Google Colab
- Libraries: scikit-learn 1.0 or higher, gensim 4.0 or higher, NLTK 3.6 or higher
NumPy, Pandas, Matplotlib

Theory

1. Text Cleaning

Text cleaning is the foundational step in any NLP pipeline that removes noise and irrelevant information from raw text data. Raw text often contains elements that don't contribute to meaning and can negatively impact model performance.

Common Text Cleaning Operations:

a) Lowercasing: Converts all text to lowercase to ensure uniformity (e.g., "Machine" and "machine" are treated as the same word).

b) Removing URLs and Email Addresses: Eliminates web links and email addresses which are typically not useful for text analysis.

c) Removing HTML Tags: Strips HTML markup from web-scraped data using regex patterns.

d) Removing Numbers: Eliminates numeric characters unless they are specifically relevant to the analysis.

e) Removing Punctuation and Special Characters: Removes non-alphanumeric characters that don't contribute to semantic meaning.

f) Whitespace Normalization: Removes extra spaces, tabs, and newlines to create clean, consistent text.

2. Lemmatization

Lemmatization is the process of reducing words to their dictionary base form (lemma) using morphological analysis. Unlike stemming, lemmatization produces valid words and considers the context.

Approaches:

a) Dictionary-based Lemmatization: Uses predefined mappings of word forms to their lemmas.

b) Rule-based Lemmatization: Applies linguistic rules to strip suffixes systematically.

c) WordNet Lemmatization: Uses the WordNet lexical database for accurate lemmatization with POS tags.

Examples:

"running" → "run"

"better" → "good"

"computers" → "computer"

3. Stop Words Removal

Stop words are common words that appear frequently in text but carry little semantic meaning. Removing them reduces dimensionality and focuses on content-bearing words.

Common Stop Words: "a", "an", "the", "is", "are", "was", "were", "in", "on", "at", "to", "for", "of", "with".

Methods:

a) NLTK Stop Words: Uses predefined lists from NLTK corpus for 16 languages.

b) Scikit-learn Stop Words: Built-in stop words list in CountVectorizer and TfidfVectorizer.

c) Custom Stop Words: Domain-specific stop words based on frequency analysis. Caution: Sometimes words like "not", "no" are important for sentiment analysis and should not be removed.

4. Label Encoding

Label Encoding converts categorical text labels into numerical values, which machine learning algorithms require. Each unique category is mapped to an integer from 0 to n-1.

Working:

Categories={Cat₁,Cat₂,...,Cat_n}→{0,1,...,n-1}

Characteristics:

- Assigns integers based on alphabetical order of categories
- Maintains consistent mapping through fit() method
- Can reverse encoding using inverse_transform()
- More memory-efficient than One-Hot Encoding for target variables

5. TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF is a numerical statistic that reflects the importance of a word in a document relative to a collection of documents. It is calculated as:

Term Frequency (TF):

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

Inverse Document Frequency (IDF):

$$IDF(t) = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents containing term } t} \right)$$

TF-IDF Score:

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

Characteristics:

- High TF-IDF: Word is important in specific documents
- Low TF-IDF: Word is common across many documents
- Creates sparse matrix representations
- Effective for text classification and information retrieval

Algorithm / Procedure:

Step 1: Import required libraries

- Import pandas, numpy for data manipulation
- Import re for regular expressions
- Import sklearn for LabelEncoder and TfidfVectorizer
- Import pickle for saving objects

Step 2: Load or create text dataset

- Create sample dataset with text and categories
- Convert to pandas DataFrame
- Display original dataset statistics

Step 3: Perform text cleaning

- Convert text to lowercase
- Remove URLs using regex pattern
- Remove email addresses
- Remove HTML tags
- Remove numbers
- Remove punctuation and special characters
- Normalize whitespace
- Display cleaning results

Step 4: Apply lemmatization

- Initialize lemmatizer (WordNet or rule-based)

- Tokenize cleaned text
- Apply lemmatization to each word
- Reconstruct text from lemmatized words
- Display lemmatization results

Step 5: Remove stop words

- Load stop words list (NLTK or custom)
- Tokenize lemmatized text
- Filter out stop words
- Reconstruct text from filtered words
- Display stop word removal results

Step 6: Encode categorical labels

- Initialize LabelEncoder
- Fit encoder on category column
- Transform categories to numeric labels
- Display label mapping
- Add encoded column to dataset

Step 7: Create TF-IDF representations

- Initialize TfidfVectorizer with parameters
- Fit and transform processed text
- Extract feature names
- Convert sparse matrix to dense array
- Display TF-IDF matrix
- Analyze top terms per document

Step 8: Save all outputs

- Save processed dataset as CSV
- Save TF-IDF matrix as numpy array
- Save fitted vectorizer using pickle
- Save fitted label encoder using pickle
- Save feature names
- Create and save processing summary

Viva Questions

1. What is text cleaning and why is it necessary?
2. What is the difference between stemming and lemmatization?
3. What are stop words? Give examples.
4. What is Label Encoding?
5. What does TF-IDF stand for?

Observation:

Text Cleaning Results:

Original Text	Cleaned Text
"Machine Learning is a subset..."	"machine learning is a subset..."
"Natural Language Processing (NLP)..."	"natural language processing nlp..."

Lemmatization Examples:

Word	Lemma
learning	learn
computers	computer
networks	network
transforming	transform

Stop Words Removal Impact:

Before	After	Words Removed
"machine learn is a subset of..."	"machine learn subset..."	is, a, of
"data science combine statistic..."	"data science combine statistic..."	and

Category	Encoded Label
AI/ML	0
Data Science	1
Deep Learning	2
Machine Learning	3
NLP	4
Programming	5

Result / Conclusion

The experiment successfully implemented a complete text preprocessing pipeline including text cleaning, lemmatization, stop words removal, label encoding, and TF-IDF representation.

Experiment No 4

Title:

To implement a complete Transformer architecture from scratch using the PyTorch library, understanding all fundamental components including multi-head attention, positional encoding, encoder-decoder architecture, and feed-forward networks.

Objective:

- To understand the Transformer architecture as proposed in "Attention is All You Need" paper
- To implement input embeddings and positional encoding for sequence processing
- To build multi-head self-attention mechanism from scratch
- To create encoder and decoder blocks with residual connections and layer normalization
- To construct a complete encoder-decoder Transformer model
- To test the model with sample inputs and validate output shapes
- To analyze the model's parameter count and architecture

Course Outcomes:

- **CO1:** Understand modern deep learning architectures for sequence-to-sequence tasks
- **CO2:** Implement attention mechanisms and transformer components using PyTorch
- **CO3:** Apply advanced neural network concepts like residual connections and layer normalization
- **CO4:** Design and build complete transformer models for NLP applications
- **CO5:** Analyze model architecture, parameters, and computational complexity

PO & PSO Mapping

PO/PSO	Description	Level
PO1	Apply deep learning fundamentals for sequence modeling	3
PO2	Design complex neural architectures using PyTorch	3
PO3	Analyze and optimize transformer model performance	3
PO5	Use modern deep learning frameworks (PyTorch)	3
PSO1	Demonstrate advanced Python and PyTorch programming	3
PSO2	Apply state-of-the-art NLP techniques	3
PSO3	Implement research paper architectures from scratch	3

Hardware Requirements:

- System: Intel Core i3 or higher processor
- RAM: Minimum 4 GB (8 GB recommended)
- Hard Disk: Minimum 20 GB free space
- Display: Standard monitor with minimum 1024×768 resolution

Software Requirements:

- Operating System: Windows 10/11, Linux (Ubuntu 18.04 or higher), or macOS

- Python: Version 3.7 or higher
- IDE: Jupyter Notebook, PyCharm, VS Code, or Google Colab
- Libraries: scikit-learn 1.0 or higher, gensim 4.0 or higher, NLTK 3.6 or higher
NumPy, Pandas, Matplotlib

Theory

1. Transformer Architecture Overview

The Transformer is a deep learning architecture introduced in the paper "Attention is All You Need" (Vaswani et al., 2017). It revolutionized sequence-to-sequence modeling by relying entirely on attention mechanisms without using recurrent or convolutional layers.

Key Advantages:

- Parallel processing of sequences (unlike RNNs)
- Better handling of long-range dependencies
- Scalable to very large models
- State-of-the-art performance in NLP tasks

2. Input Embeddings

Input embeddings convert discrete token indices into continuous vector representations. Each token is mapped to a dense vector of dimension d_{model} .

Formula:

$$\text{Embedding}(x) = \text{EmbeddingLayer}(x) \times \sqrt{d_{model}}$$

The scaling by $\sqrt{d_{model}}$ helps stabilize gradients during training.

3. Positional Encoding

Since Transformers don't have inherent sequence order (unlike RNNs), positional encodings are added to embeddings to inject position information.

Sinusoidal Positional Encoding:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

where:

- pos = position in sequence
- i = dimension index
- d_{model} = embedding dimension

Properties:

- Uses sine for even indices, cosine for odd indices
- Allows model to learn relative positions
- Can extrapolate to longer sequences than seen during training

4. Multi-Head Attention

Multi-head attention allows the model to jointly attend to information from different representation subspaces.

Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

where:

- **Q**= Query matrix
- **K** = Key matrix
- **V** = Value matrix
- D_k = dimension of keys

Multi-Head Attention:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

where h is the number of attention heads.

Benefits:

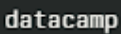
- Captures different types of relationships
- Increases model expressiveness
- Parallel computation across heads

5. Feed-Forward Networks

Position-wise feed-forward networks apply the same fully connected layers to each position independently.

Formula:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Typically $d_{ff} = 4 \times d_{model}$ (e.g., 2048 when $d_{model} = 512$). 

6. Layer Normalization

Layer normalization normalizes inputs across the feature dimension, stabilizing training.

Formula:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

where:

- μ = mean across features
- σ^2 = variance across features
- γ, β = learnable parameters
- ϵ = small constant for numerical stability

7. Residual Connections

Residual (skip) connections add the input of a sub-layer to its output, helping gradient flow.

Formula:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

8. Encoder Architecture

The encoder consists of NN identical layers, each with:

1. Multi-head self-attention mechanism
2. Position-wise feed-forward network
3. Residual connections around each sub-layer
4. Layer normalization

9. Decoder Architecture

The decoder consists of NN identical layers, each with:

1. Masked multi-head self-attention (prevents attending to future positions)
2. Multi-head cross-attention (attends to encoder output)
3. Position-wise feed-forward network

4. Residual connections and layer normalization

10. Complete Transformer

The complete model combines:

- Source and target embeddings with positional encodings
- Encoder stack (N layers)
- Decoder stack (N layers)
- Final linear projection to vocabulary size
- Softmax for probability distribution

Algorithm / Procedure:

Step 1: Import required libraries

- Import PyTorch and torch.nn
- Import math for mathematical operations
- Import numpy for array operations

Step 2: Define Input Embedding layer

- Create Embedding layer for vocabulary
- Scale embeddings by $\sqrt{d_{\text{model}}}$

Step 3: Implement Positional Encoding

- Create positional encoding matrix using sine/cosine
- Register as buffer (non-trainable)
- Add to input embeddings

Step 4: Implement Layer Normalization

- Calculate mean and standard deviation
- Normalize and apply learnable scale/shift

Step 5: Build Feed-Forward Network

- Create two linear layers
- Apply ReLU activation between layers
- Add dropout for regularization

Step 6: Implement Multi-Head Attention

- Create Query, Key, Value projection matrices
- Split into multiple heads

- Apply scaled dot-product attention

- Concatenate heads and project

Step 7: Create Residual Connection module

- Apply layer normalization

- Execute sublayer

- Add residual connection with dropout

Step 8: Build Encoder Block

- Add self-attention with residual connection

- Add feed-forward with residual connection

Step 9: Stack Encoder Blocks

- Create N encoder blocks

- Apply final layer normalization

Step 10: Build Decoder Block

- Add masked self-attention

- Add cross-attention with encoder output

- Add feed-forward network

- All with residual connections

Step 11: Stack Decoder Blocks

- Create N decoder blocks

- Apply final layer normalization

Step 12: Create Projection Layer

- Linear layer to project to vocabulary size

Step 13: Assemble Complete Transformer

- Combine all components

- Initialize parameters (Xavier initialization)

Step 14: Test the model

- Create dummy inputs

- Perform forward pass

- Verify output shapes

- Count parameters

Viva Questions

1. What is the Transformer architecture and why is it important?
2. What is the purpose of positional encoding in Transformers?
3. What does "attention" mean in the context of neural networks?
4. How many layers are in the standard Transformer encoder and decoder?
5. What is the dimension of the model (d_model) in the original paper?

Observation:

Component	Value	Description
d_model	512	Model embedding dimension
N (layers)	6	Number of encoder/decoder layers
h (heads)	8	Number of attention heads
d_ff	2048	Feed-forward hidden dimension
d_k	64	Dimension per attention head (512/8)
Dropout	0.1	Dropout probability

Tensor Shapes Through the Model:

Layer	Input Shape	Output Shape
Input Embedding	(batch, seq_len)	(batch, seq_len, 512)
Positional Encoding	(batch, seq_len, 512)	(batch, seq_len, 512)
Multi-Head Attention	(batch, seq_len, 512)	(batch, seq_len, 512)
Feed Forward	(batch, seq_len, 512)	(batch, seq_len, 512)
Encoder Output	(batch, src_seq, 512)	(batch, src_seq, 512)
Decoder Output	(batch, tgt_seq, 512)	(batch, tgt_seq, 512)

Layer	Input Shape	Output Shape
Projection	(batch, tgt_seq, 512)	(batch, tgt_seq, vocab_size)

Result / Conclusion

The experiment successfully implemented a complete Transformer architecture from scratch using PyTorch, following the "Attention is All You Need" paper.

Experiment No 4

Title:

To study and understand the concepts of morphology in Natural Language Processing through the implementation and analysis of Add-Delete tables for word transformation.

Objective:

- To understand morphology as the study of word structure and formation
- To identify morphemes (smallest meaningful units) in words
- To implement Add-Delete table operations for morphological analysis
- To analyze morphological transformations (prefixes, suffixes, roots)
- To distinguish between inflectional and derivational morphology
- To compute and visualize all possible add and delete operations on words

Course Outcomes:

- CO1: Understand fundamental concepts of morphological analysis in NLP
- CO2: Implement computational methods for word structure analysis
- CO3: Analyze word formation patterns using systematic operations
- CO4: Apply morpheme identification techniques to natural language
- CO5: Evaluate morphological complexity through add-delete operations

PO & PSO Mapping

PO/PSO	Description	Level
PO1	Apply linguistic concepts to computational problems	3
PO2	Design algorithms for morphological analysis	3
PO3	Analyze word structure systematically	2
PO5	Use Python for linguistic data processing	3
PSO1	Demonstrate programming skills for NLP tasks	3
PSO2	Apply computational linguistics techniques	3

Hardware Requirements:

- System: Intel Core i3 or higher processor
- RAM: Minimum 4 GB (8 GB recommended)
- Hard Disk: Minimum 20 GB free space
- Display: Standard monitor with minimum 1024×768 resolution

Software Requirements:

- Operating System: Windows 10/11, Linux (Ubuntu 18.04 or higher), or macOS
- Python: Version 3.7 or higher
- IDE: Jupyter Notebook, PyCharm, VS Code, or Google Colab
- Libraries: string (built-in), collections (optional)

Theory

1. Morphology

Morphology is the study of the internal structure of words and how words are built from smaller meaningful units. It is a fundamental component of linguistic analysis and natural language processing.

Definition: Morphology examines how morphemes combine to form words and how these combinations affect meaning and grammatical function.

2. Morphemes

A morpheme is the smallest meaningful unit in a language. Morphemes cannot be divided into smaller meaningful parts.

Types of Morphemes:

a) Free Morphemes: Can stand alone as words

- Examples: cat, book, happy, run

b) Bound Morphemes: Must attach to other morphemes

- Examples: -ing, -ed, un-, re-, -ness

c) Root/Base Morphemes: Core meaning-bearing unit

Examples: walk (in walking), happy (in unhappy)

d) Affixes: Morphemes attached to roots

- Prefix: Attached before root (un-, re-, pre-, dis-)
- Suffix: Attached after root (-ing, -ed, -ness, -ly)
- Infix: Inserted within root (rare in English)

3. Types of Morphology

a) Inflectional Morphology: Changes grammatical function without changing word class or core meaning

- Plural: cat → cats
- Tense: walk → walked, walking
- Comparison: big → bigger, biggest
- Person: walk → walks (3rd person singular)

b) Derivational Morphology: Creates new words or changes word class

- happy (adjective) → happiness (noun)
- write (verb) → rewrite (verb)
- care (noun) → careful (adjective) → carefully (adverb)

4. Add-Delete Table

An Add-Delete table is a computational tool for understanding morphological transformations by systematically applying character addition and deletion operations.

Operations:

ADD Operation: Insert a character at any position in the word

1. Formula: $(n+1) \times |\Sigma| (n+1) \times |\Sigma|$ operations
2. where n = word length, $|\Sigma|$ = alphabet size (26 for English)
3. Example: "cat" → "cats" (add 's' at end)

DELETE Operation: Remove a character from any position

- Formula: n operations
- where n = word length
- Example: "cats" → "cat" (delete 's')

Total Operations:

Total = $(n+1) \times 26 + n$ Total = $(n+1) \times 26 + n$

5. Morphological Analysis in NLP

Morphological analysis breaks words into constituent morphemes and identifies their grammatical roles.

Applications:

1. Lemmatization: Reducing words to base form (running → run)
2. Stemming: Crude chopping to approximate root
3. POS Tagging: Morphological features help identify parts of speech
4. Information Retrieval: Search different word forms
5. Machine Translation: Understanding word structure aids translation
6. Text-to-Speech: Morphological parsing improves pronunciation

Example Analysis:

- Word: "unhappiness"
- Morphemes: un- (prefix) + happy (root) + -ness (suffix)
- Meaning: state of being not happy

6. Morphological Parsing

Morphological parsing is the process of analyzing a word form to identify its morphemes and their grammatical properties.

Approaches:

- Rule-based: Uses linguistic rules and affix dictionaries
- Statistical: Machine learning models trained on annotated data
- Hybrid: Combines rules with statistical methods

7. Edit Distance and Morphology

Add-Delete operations relate to edit distance (Levenshtein distance), measuring the minimum operations needed to transform one word to another.

This concept is fundamental in:

- Spell checking
- Morphological generation
- Word similarity computation
- Fuzzy string matching

Algorithm / Procedure:

Step 1: Define morphological concepts

- Identify types of morphemes (root, prefix, suffix)
- Understand inflectional vs derivational morphology

Step 2: Implement ADD operation function

- For each position in word (0 to length)
- For each letter in alphabet (a-z)
- Insert letter at position
- Store operation and result

Step 3: Implement DELETE operation function

- For each position in word (0 to length-1)
- Remove character at position
- Store operation and result

Step 4: Create Add-Delete table display function

- Generate all ADD operations
- Generate all DELETE operations
- Format and display in tabular form

Step 5: Implement morpheme identification

- Define common prefixes and suffixes
- Check word for prefix match
- Check remaining for suffix match
- Identify root morpheme

Step 6: Analyze morphological transformations

- Compare base word with target word
- Identify required ADD/DELETE operations
- Map operations to morphological processes

Step 7: Calculate statistics

- Count total operations for different word lengths
- Verify formula: $(n+1)*26 + n$
- Display comparative analysis

Step 8: Test with sample words

- Apply to words of varying complexity
- Analyze morphological structure
- Document results

Viva Questions

1. What is morphology in linguistics?
2. What is a morpheme? Give examples.
3. What is the difference between free and bound morphemes?
4. What is a prefix? Give three examples.
5. What is a suffix? Give three examples.
6. What is the root or base of a word?

Result / Conclusion

The experiment successfully demonstrated morphological analysis using Add-Delete tables as a computational tool.